Biosignal based Side Channel Attack to Infer Android Pattern Lock Using Deep Learning

Arun Sarma
University of Washington Bothell
asarma@uw.edu

Geethapriya Thamilarasu
University of Washington Bothell
geetha@uw.edu

***Abstract*—The growing popularity of wearable Internet of Things (IoT) devices has led to significant security and privacy concerns. The health data that these devices collect can be used to infer private and sensitive user information via side channel attacks. This is especially true for users of Brain-Computer Interfaces (BCI) which measures brain activity via Electroencephalography (EEG) signals, and sometimes muscle movement from Electromyography (EMG) signals in Human-Computer Interaction (HCI) applications. Studies show attacks have been constructed to infer various sensitive information such as PINs and passwords from BCI users' biosignal data. However, to our knowledge, no side channel attacks have been demonstrated on popular, alternative authentication methods such as Android Pattern Lock. Existing research shows that Android Pattern Lock can be cracked using video-based and acoustic side channel attacks. However, these attacks require direct observation of the victim or access to the victim's smartphone sensors which can be locked behind strict device permissions. Motivated by the vulnerabilities of consumer-grade IoT devices that record biosignal data, we propose a novel side channel attack where recorded EEG and EMG signals are analyzed using deep learning techniques to infer a victim's Android Pattern Lock. Our experiment results show that our side channel attack detects when a user is unlocking their phone via Pattern Lock with a 96.97% accuracy and infers the drawn pattern with a 99.97% accuracy. General swipe directions of the user's finger drawing the unlock pattern is inferred with a 93.64% accuracy.**

I. Introduction

In recent years, wearable Internet of Things (IoT) devices for personal health have grown in popularity for their ability to provide health insights or new user experiences to users. These devices such as smartwatches provide user experiences by measuring body health metrics and are continuously updated with new sensors. Some of these devices such as smart headbands and arm or wristbands have also been used in enhanced Human Computer Interaction (HCI) systems. Specialized devices such as Brain-Computer Interfaces (BCIs) measure brain activity via Electroencephalography (EEG) signals from individuals to offer insights into their mental health. BCI devices such as the EEG headset, are growing in popularity for their accessibility in the consumer market. They pair with BCI software to provide users with details on their physiological state such as emotions, focus, stress level. These devices can allow for more immersive experiences where users can control applications such as video games, or physical peripherals such as robots, or prosthetics just by thinking of a command or physical action [16]. In addition to EEG, Electromyography (EMG) is a biosignal which has traditionally been used in a medical setting to measure muscle activity but has potential applications in consumer devices. Advanced HCI systems can be made with embedded EMG sensors, allowing users to control peripherals by muscle activity alone. EMG electrodes can already be easily integrated into smartwatch wristbands which can allow individuals to conduct their own muscle therapy or monitor their muscle conditions [4]. Future Virtual Reality (VR) devices can be controlled by using EMG wristbands to detect finger gesture alone, without the need for controllers with joysticks and buttons [21].

While BCI and advanced HCI devices that utilize biosignals are beneficial from a health, accessibility, and entertainment standpoint, they also present significant privacy and security risks. The commonality between all these devices is that they are meant to be worn for extended periods of time, where they continuously collect users' biosignal data. The data collected by these devices can potentially be used to expose private and sensitive user information via side channel attacks. For instance, if EEG signals are intercepted from an EEG headset, then user-sensitive information (such as emotional state, personal preferences, thoughts) can possibly be inferred and exploited for malicious purposes [22]. This threat becomes more severe if the user is performing an activity that involves sensitive information, such as logging into their bank account, while wearing an IoT device such as an EEG headset. Raw EEG data intercepted at that time could be processed and fed through a deep learning model to reveal sensitive information pertaining to that activity. Furthermore, side channel attacks can be used to expose information from other biosignal data such as EMG data. Data from benign HCI systems that utilize EMG to decode finger movement, can be intercepted and used to expose a variety of keyboard input such as passwords with promising accuracy [9]. Taking this into consideration, it is important to bring attention to these potential security vulnerabilities and if possible, any existing mitigation strategies.

The goal of our work is to propose a side channel attack where malicious access to EEG and EMG signals captured by consumer-grade devices can be used to reveal a type of sensitive information from the users of these devices. These users may be wearing the BCI or advanced HCI device as they would a smartwatch or a pair of headphones, while using their smartphone. They may conduct operations involving sensitive

information such as unlocking their phone or a particular application. If a malicious app is capturing the EEG and EMG signals when users are entering their log-in information, their log-in information could be revealed via signal analysis and machine learning techniques. Hence, we propose a side channel attack which infers smartphone log-in information from EEG and EMG biosignals by processing the raw signals and training a deep learning model.

II. Background & Prior Work

A majority of current BCI research focuses on how algorithms or machine learning models can be trained on databases of EEG signals or other biosignals to measure human state and predict the presence of mental conditions. For instance, BCIs can evolve to become neurofeedback loops to read and stimulate parts of the brain in response to how the user is feeling for rehabilitation purposes [32]. In addition, BCIs may have applications in military, investigations, or the workplace where various malicious actors are already active [16]. It has been proven by various studies that consumer-grade BCI devices can also be used to decode imagined speech, which was previously usually done solely by research-grade EEG recording apparatus. Before these abilities are realized, the ability to secure these systems is paramount since there are severe consequences if advanced BCIs with unfiltered access to people's minds are compromised.

A. Using Biosignals for Inference

BCIs infer user activities by recording EEG, a type of biosignal. Biosignals are signals that are heavily used in biomedicine as the primary source of information about biological systems. Biosignals have been also been used in combination to infer information about the body. For example, EEG and Electrocardiogram (ECG) are two significant measurements that can be used together to detect the presence of heart disease with very high accuracy [27]. Both EEG and EMG signals are often used in studies to classify hand and finger positions, and used in applications to control robotic arms and prosthetics [26], [33]. EMG sensors are accessible and can be integrated into existing systems using compact sensor systems [5] such as smartwatches. EEG signals are primarily used in BCI devices to infer various user information such as emotions, thoughts, and other brain-related activity. However, EEG is noise prone. By measuring other biosignals and EEG simultaneously, time spans of data where unrelated physiological activity occurs can be identified and filtered out. D. -Y. Kim et al. proposed a framework to classify driver drowsiness by analyzing EEG signals while avoiding the variability issues of EEG [15]. Machine learning techniques are often used in biosignal-based inference tasks. For example, Spampinato [31] studied how to use Long-Short Term Memory (LSTM) cells to translate multichannel EEG signals into a 1-D feature vector to enhance computer vision models to enhance a CNN based image classification system. This work highlights the potential of using deep learning techniques to process EEG signal data to infer human vision. Deep learning approaches are also used to classify activities based on EEG signals. The benefit of using deep learning architecture is that they can learn on raw or minimally processed EEG data and achieve strong results on EEG classification tasks [29]. Voting algorithms have also been proposed to efficiently select machine learning models which can analyze the EEG data with higher accuracy despite high noise or non-linearity [8]. CNNs and other neural networks can be enhanced to be shift invariant via the use of functional data analysis to outperform standard EEG classification models like EEGNet [10]. EEG and EMG signals can be used together to classify various physical actions [13]. This research demonstrates the ability to combine features from biosignals and sensors measuring physical movement to train standard classifiers and machine learning models with good performance. EMG alone was used to build a system that unlocks a smartphone based on how the user picks up the device with high accuracy [7]. In this research a Siamese Network is used for one-shot learning of the EMG signals corresponding to the action of picking up a smartphone. Similar to how EEG signals are noise-prone due to various features in the recording environment, EMG signal accuracy can be influenced by muscle anatomy where multiple muscles can activate for a single action. Targeting specific muscle groups when measuring EMG data helps differentiate finer grain muscle activities. This is demonstrated by various studies which classify hand and finger gestures with good accuracy since multi-channel EMG was used to measure from various muscles [11], [19], [26], [38].

B. IoT Device Security Risks

The security of these devices and systems has gained greater importance due to the rise in popularity of personal health devices. The IoT space has a projected market size of 265.4 billion dollars (USD) in 2026, and one of the IoT challenges foreseen is the lack of standardized security mechanisms for these devices that carry personal information [4]. With these IoT devices, there are many parts of their systems that can be compromised to allow hackers access to user's private information. Mandal et al. details how data can be intercepted via malicious smartphone apps, from dependencies such as cloud storage services, or from intercepting the signal in transit from the user device [22]. In addition, various studies present various scenarios where systematic attacks against wearable health related IoT devices have revealed vulnerabilities in products used by consumers. For example, Eberz [6] demonstrated that a user's biometrics, such as ECG and eye movements, can be inferred by other fitness data gathered in a different context. The same study showed how the Nymi Band ECG biometrics device can be compromised. Xiao mentioned in their study how the ANT protocol for Fitbit data communications was reverse engineered to generate various attacks, and the radio protocol for a popular glucose monitoring and insulin delivery system was recovered to allow for various attack scenarios [34].

C. BCI Security Risks

A subset of BCI and EEG research papers focus on demonstrating the security threats for BCI and other advanced HCI systems. The goal of these papers specifically, is to show how EEG can be used maliciously to infer user activity. BCI hacking can cause a compromise in an individual's personal autonomy (by having an algorithm in between human thoughts and inferences), and privacy and security (by eavesdropping on personal thoughts, monitoring attentiveness in the workplace, leaking of mental data, etc.) [16]. Martinovic explored how an EPOC EEG headset can be used to infer private information about their users [5]. Kapitonova [14] detailed how current BCI technology allows for the detection of implicit associations and the inference of bank information, date of birth, or geographical location of the user. Apart from revealed sensitive information, a related consequence of a compromised BCI system is personal autonomy as well. King [16] mentioned eavesdropping of personal thoughts, identification of people, monitoring of attentiveness at work, or even manipulation of EEG test results as the most referenced threats in BCI research. These threats are not far-fetched as recent papers have demonstrated similar abilities in their simulations. For example, Agarwal [1] demonstrated how phishing advertisements can be created to infer user preferences and their associations from measuring the Event-Related Potentials (ERP), which can reveal significant reactions to stimuli, such as when the subjects were presented with images of familiar places. For neuromarketing systems, eye-tracking, facial coding, Galvanic Skin Response (GSR) data can be used separately or be paired with EEG data to create a more detailed inference on consumers' reactions to certain products in a marketing study [2]. A similar attack can be used to expose user's health conditions. For example, EEG and Electrocardiogram (ECG) are two significant measurements that can be used together to detect the presence of heart disease with very high accuracy [27].

Another interesting threat is related to neurocrime, where not only do hackers get access to brain activity information, but also can limit, modify, or disrupt BCI peripherals. For example, hacking health BCIs that allow users to control prosthetics, robotic limbs, or other critical hardware. Ienca [18] showed that brain hacking is possible by building software that either learns corresponding activities from the user's EEG signals to generate false ones, adding noise to a user's EEG signals to make decoding impossible, or by directly stopping the regular functionality until the user provides personal information. Xiao [34] not only revealed the process for intercepting EEG signals from a consumer-grade EEG headset using various attack vectors, but also demonstrated a deep learning model that can be used to infer user activity. This model consisted of combining the outputs from separate RNN and CNN models using a feed-forward network. They trained this model on a dataset from Neurosky which consisted of human activities such as reading instructions, solving math problems, thinking of an item, blinking, etc. and achieved a significantly high accuracy relative to prior models. While this study made great strides in developing a model to train on a EEG dataset of human activity, the activities inferred did not directly contain sensitive information. Neupane et al. [25], showcased how subjects' PINs can be predicted by using simple classifiers trained on a dataset containing EEG signals of participants as they typed randomly generated PINs. While this study greatly improved the classification accuracy of keystrokes using EEG signals over previous studies, deep learning techniques may have improved their results for a true-positive rate of higher than .50. Mishra et al. [24] created a 1-D CNN model that classified, with good accuracy, the digits (0-9) that a person was thinking of. Standard EEG feature extraction methods were used as well as dimensionality reduction and clustering methods to visualize the grouping of EEG signal samples. Select papers have focused on strategies to guard against BCI attacks. The methods range in feasibility based on their application. For example, Shahzadi [30] proposed a method to use optical chaos to add random noise to the EEG signal before being transmitted for remote analysis in a medical setting. Li [20] proposed an approach of transmitting the minimum EEG features or channels of EEG signals for analysis instead of the full raw EEG signal. This method scopes the amount of information that can be extracted so that the likelihood of side-channel attacks is lessened.

D. Android Pattern Lock

Android Pattern Lock is a popular method of signing into Android smartphones and is an alternative to PIN and password entry systems. Pattern Lock works by having the user draw a pattern on a 3x3 grid, where the pattern has at least 4 nodes and each node is only visited once. Vertical, horizontal, and diagonal line segments are allowed in the pattern. A pattern is represented as a lock sequence of numbers representing the order in which the nodes were visited. For instance, Figure 1 shows a pattern of 012587436, where the start node is at the top left of the grid.

1) Android Pattern Lock Security Threats: The Android Operating System encrypts users' pattern lock sequences with the SHA1 hashing algorithm and stores it in a gesture.key file in the file directory which can only be accessed if the device is rooted and has developer options enabled [28]. A hacker could crack Android Pattern Lock if they have a dictionary of all possible lock sequences and their hash values and look for the lock sequence that matches the hash on the victim's device. As mentioned by the article, this is not feasible without the device being rooted with developer options enabled. Most causal Android users probably don't have Android devices that meet those two conditions.

2) Side-Channel Attacks: A few studies have explored side-channel methods of cracking the Pattern Lock and gain access to a user's smartphone. Video-based side channel attacks use a computer vision algorithm reconstruct the pattern from a user's finger movements on their screen [35]. However, this approach is reliant upon having a video of the victim of good enough quality for the computer vision algorithm to predict

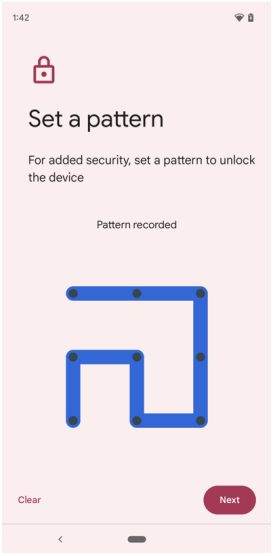

Fig. 1. An example of Android Pattern Lock

a pattern. An acoustic side channel attack was proposed to infer a locking pattern by analyzing sound wave reflections off the user's finger as it slides across their screen [37]. Their approach is dependent on microphone access being enabled for background processes. However, this functionality has since been restricted to apps running in the foreground or services that push a notification to notify the user that the microphone is in use. Another way for Pattern Lock to be exposed is by analyzing how WiFi signals are influenced by the user's finger motions as they draw the pattern [36]. The limitations of this approach are that other moving objects or body parts can interfere with or disrupt the WiFi signal. Lastly, a user's oily residue or smudge from their fingers traveling on the screen can be used to retrieve the pattern [3]. However, this method relies on the smudge being preserved for the attacker to observe it before other finger movements distort or remove it from the screen.

E. Our Contribution

In contrast to the mentioned works, we propose a novel threat model that uses both the EEG and EMG signal to detect the user is unlocking their phone via Pattern Lock and to infer the pattern they are drawing. We show how a potential malicious BCI/HCI device or application, recording EEG and/or EMG data can be used to elicit users' sensitive information such as their smartphone log-in information.

III. Threat Model

An attacker's motivation for executing a biosignal-based side-channel attack can be to reveal sensitive information that a user has entered on a device like their computer or smartphone.

A. Biosignal-Based Side-Channel Attacks

These side-channel attacks can be implemented with the help of deep learning models which analyze related or incidental data to infer the targeted sensitive information. Deep learning and machine learning solutions have been proposed to infer sensitive information such as keystrokes from a full-size keyboard using EMG [9] and personal identification numbers (PINs) using EEG [25]. Malicious BCI apps can be developed to capture EEG signals and keystrokes, unbeknownst to the wearer of the EEG headset. Xiao [34] demonstrated the feasibility of this by exploiting a standard SDK for a consumer-grade EEG headset. Similar capabilities can be extended to EMG-recording devices, where attackers covertly access user's EMG data and infer hand gestures corresponding to a known user-interface [9]. This type of threat model assumes the user has been fooled into installing a malicious app that interfaces with BCI or EMG devices.

B. Using Biosignal Side-Channels to Infer Android Pattern Lock

In our study, the attacker's motive is to covertly intercept a victim's EEG and EMG signals, recorded by wearable IoT devices to seek sensitive information such as their Android Pattern Lock. The attacker accesses user biosignals from wearable IoT devices such as BCIs via installation of malicious software as described by previously explored studies [1], [16], [23], [25], [34]. The victim's unlock pattern is inferred from processing the EEG and EMG signals recorded while the victim draws their unlock pattern. The mental activity related to drawing the unlock pattern, and the muscle extension/contraction activity is reflected in the temporal features of the EEG and EMG signals, respectively.

Our work is likely among the first to explore the side-channel attack of using EMG and EEG sensor data to infer Android Pattern Lock. Prior attempts to decode Pattern Locks used an acoustic side-channel attack, Computer Vision (CV) techniques or investigated security vulnerabilities in the Android OS itself [28], [35], [37].

IV. The Proposed Approach

To demonstrate the threat model, we propose an experiment to determine how a user's smartphone Pattern Lock can be exposed by analyzing their EEG and EMG signals. The targeted scenario is that a user is wearing a EEG headset and EMG wristband for health or entertainment purposes, where the biosignal data is being transmitted to a BCI or health-tracking application for processing. We consider a side-channel attack where the user's EEG and EMG data is covertly recorded by a hacker by exploiting a vulnerability in the application while the user is actively typing/drawing their Pattern Lock. Through this research, we aim to demonstrate the feasibility of inferring user login information by intercepting EEG and EMG biosignals using deep learning models.

Our threat model of inferring Pattern Lock involves inferring when the user is unlocking their phone with Pattern Lock and also inferring the pattern drawn during the Pattern Lock activity. These inferences are made by training deep learning models on recorded EMG and EEG signals to classify the current user activity and their drawn pattern. Attainable, consumer grade EEG and EMG recording hardware is used for

data collection. The EEG headset records data from locations spread across the temporal and frontal lobes of the subject. For recording EMG, consumer-grade EMG electrodes are placed on the wrist, underneath the base of the thumb. We chose the thumb since we assume the user is using their thumb to draw the unlock pattern.

The overall inference problem is split into two smaller inference steps. The first step is Pattern Lock detection where the EEG signal is used to determine if the user is actively unlocking their phone via Pattern Lock activity or doing another random phone activity like scrolling or typing. The EEG signal recorded during a Pattern Lock activity reflects a difference in mental activity and focus required versus other common phone activities. The second scenario is Pattern Lock identification where the EMG signal is used to determine which pattern the user memorized and drew to unlock their phone. The EMG electrodes target the flexor pollicis longus which is the muscle responsible for thumb extension [12]. Thus, the recorded EMG signals carry information about muscle activity related to the thumb movements.

Two approaches are tested for Pattern Lock identification. In the first approach, we attempt to classify a set of predetermined Android Pattern Lock patterns. As there are many possible pattern types, this study focuses on inferring predetermined Pattern Locks. 6 patterns are randomly chosen from the existing dataset of Pattern Locks [35], where there is an equal amount from each complexity type (Simple, Medium Complex, and Complex) as shown in Figure 2. The activity of drawing the Pattern Lock is only constrained by the time limit of 6 seconds. A single recording session has multiple recordings for various pattern types.

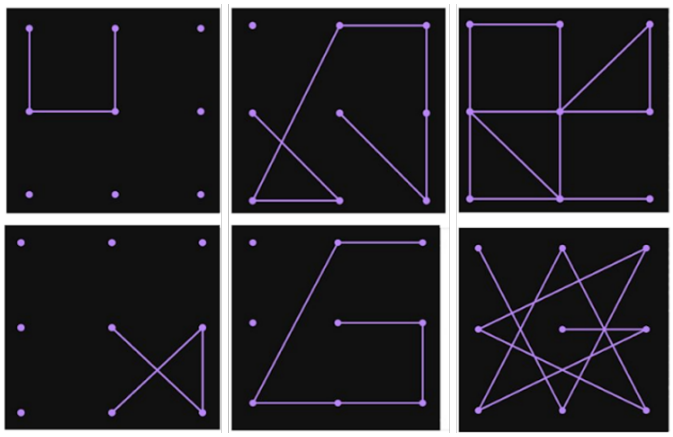

Fig. 2. Targeted Pattern Locks with corresponding sample EMG signals, ranging from simple (left) to moderately complex (middle) to complex (right)

In the second approach, we attempt to classify all 16 possible vectors that a user can draw on the standard 3 x 3 Pattern Lock grid, see Figure 3. The purpose of this approach is to showcase how an eventual system can be created to infer almost all possible patterns by recognizing each line segment direction of the pattern to reconstruct the pattern that the user drew.

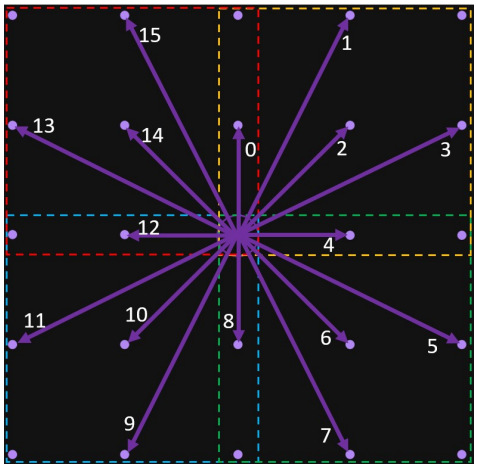

Fig. 3. All 16 draw-able vectors Android Pattern Lock, labelled from 0 - 15

A. Experiment Setup

The experiment of inferring Android Pattern Lock is split into two separate experiments as described earlier and shown in Figure 4. The subject conducts two separate activities. The first activity is random phone activity: scrolling, swiping or typing, which does not require any memorization. The second activity is unlocking the smartphone via Pattern Lock which requires the users to memorize a set of patterns to draw at the time of recording as described earlier. There are three pattern types: Simple, Moderate, Complex chosen at random for use in our experiment. These pattern types are meant to encompass three pattern complexities that can be seen in a real-world setting, Simple Moderate and Complex as shown in Figure 2.

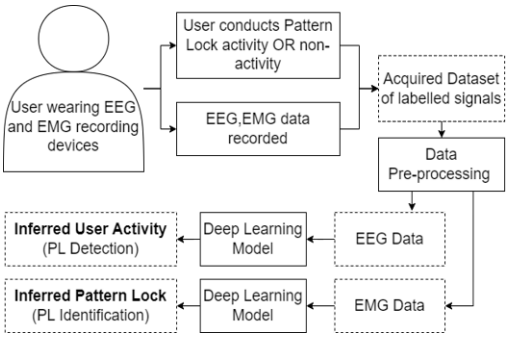

Fig. 4. Side Channel Attack Architecture

B. Data Collection

To collect biosignal data, we create a system that simultaneously records EMG and EEG data while the user is drawing a Android Pattern Lock as mentioned in Figure 4. Both the EEG and EMG sources record timestamps, thus it is relatively straightforward to sync the two signal data into a combined EEG&EMG recording sample. The ground truth for the recordings is set on the laptop via software which

orchestrates the data collection session. This software records and labels samples to construct our dataset. In our study, we collected data from a single participant but the same collection methods can be scaled to include other participants.

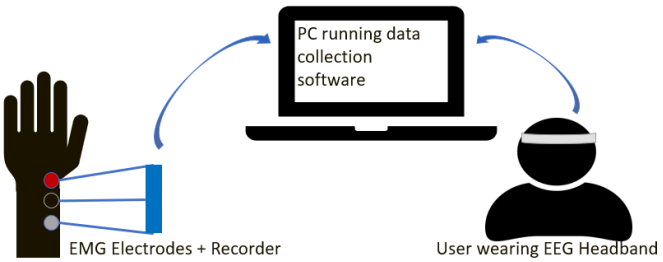

Fig. 5. High-level data collection setup

The subject sits in a chair with a phone in their right hand with our custom Pattern Lock simulator application open. This simple application displays the 3x3 dot grid used in the standard Android Pattern Lock interface. Users are to draw the pattern with their thumb and thus, the EMG electrodes are placed near the thumb extension muscle, the flexor pollicis longus [12] as shown in Figure 5.

Since EEG data is recorded, the user must sit still with minimal noise and visual distractions to prevent extra noise in the EEG recording. The user memorizes the set of patterns needed for the next recording session. At the start of the session, the custom recording software runs, and displays when to start drawing the pattern to the user. The user then draws the pattern on the simulator app while the data collection software records the user's EMG and EEG data. After 5 seconds, the software saves the recorded biosignal data, pauses for 2 seconds, then prompts the user to begin drawing the next pattern. This cycle is repeated for the number of samples desired for that session. The amount and targeted patterns can be adjusted to be different for each session.

Data from both of the activities is recorded over a period of weeks. The resulting dataset of biosignals consists of separate directories for each label containing recording files. Each file is an individual sample related to the smartphone activity (drawn Pattern Lock, drawn Pattern Lock vector, or non Pattern Lock). Each individual sample is roughly the same size at 6 seconds and includes data from just before and during the observed smartphone activity. This is done to include not only the activity of drawing the pattern but also the brain and muscle activity generated when the user thinks and prepares to draw the Android Pattern Lock.

C. Data Processing & Feature Extraction

The data is first cleaned before being processed. Our criteria for valid data is that each sample is 5-seconds long and contains valid, non-null float values. After cleaning, we process the raw EMG and EEG data from our dataset in a sequence as shown in Figure 6. For EEG data, we first first filtered out the 60Hz. power-line noise. We then extracted the Beta frequency band of the EEG signals by applying a band-pass filter between 13 and 30Hz. We target the the Beta frequency band since it represents when the user is in an awake, active state thus it contains signal data most relevant to the activity of unlocking a phone. No further feature extraction was done since we use deep learning models which can achieve strong results on filtered raw EEG data. For EMG data, we normalized the samples to values between 0 and 1 to bring the data into a common scale and prevent skewing due to peak amplitude differences. The sets of processed EEG and EMG signals are compiled into separate data structures for use in the training of the deep learning models.

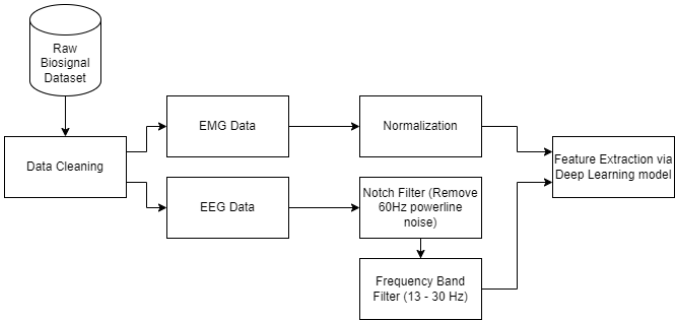

Fig. 6. EEG and EMG Data Processing Flow Chart

V. Classification Models

In this section we describe the deep learning models trained on the pre-processed data to infer the drawn patterns. The inference is formulated as three separate supervised classification problems. The first is for Pattern Lock detection where binary classification is used to infer whether the user is doing a Pattern Lock activity via EEG signals. In this problem the two classes are Pattern Lock Activity or Not Pattern Lock Activity. The second and third problems are for the two Pattern Lock identification scenarios, where multi-class classification is used to inferring the pattern drawn from the EMG signals. The first Pattern Lock identification scenario features 6 classes representing each Pattern Lock and the second scenario features 16 classes representing each possible vector draw-able on the 3 x 3 Pattern Lock grid.

The input data to the classification problems are the segments of biosignal data coinciding with the act of drawing the Pattern Lock. In case of the first inference problem of Pattern Lock detection, EEG samples corresponding with actions that are not related to Pattern Lock such as random scrolling, or swiping are also included. Each segment is approximately 5 seconds long regardless of the complexity of the pattern drawn and includes 500ms of rest state at the start.

A. Model Architectures

We implement and train various models for each of the Pattern Lock inference problems. We chose to exclusively implement and compare deep learning architectures, since they have the ability to learn hidden features from data. Thus, our side-channel attack bypasses intricate, manual feature

engineering as the model can learn feature representations from raw biosignal data. The input for each model is either the EEG signal for Pattern Lock Detection or the EMG signal for Pattern Lock Identification. Thus, the input shape to all three models is determined by the number of biosignal data channels. In our attack, the EEG data has multiple channels for each of the targeted brain regions whereas EMG data has one channel since a single muscle is targeted.

- **1D CNN**: The 1-Dimensional Convolutional Neural Network (1D CNN) is a deep learning model architecture that process input data via the use of sliding kernels. For highly dimensional data such as EEG signals, these models are useful in extracting features and reducing the dimensions of the input signal data. We use a set of 1D CNN layers with MaxPooling and Dropout layers in between. This is followed by a fully connected layer and sigmoid activation function.

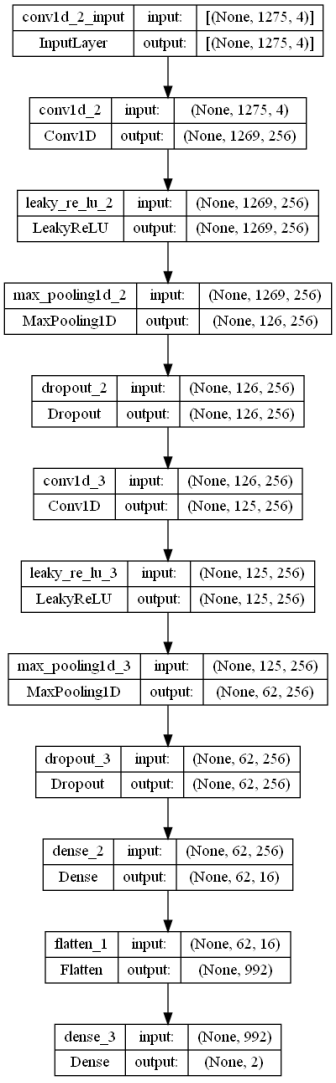

Fig. 7. 1D CNN architecture where the input shape varies by biosignal. EEG: (length of sample, # of channels) EMG: (length of sample, 1)

- **LSTM**: The Long Short-Term Memory (LSTM) models are a type of Recurrent Neural Network (RNN) designed to learn the temporal features of sequential input data. The LSTM model can learn learning useful and non-useful features of the biosignal data. We use a set of LSTM and Dropout layers followed by a fully connected layer and softmax activation function for multi-class classification.

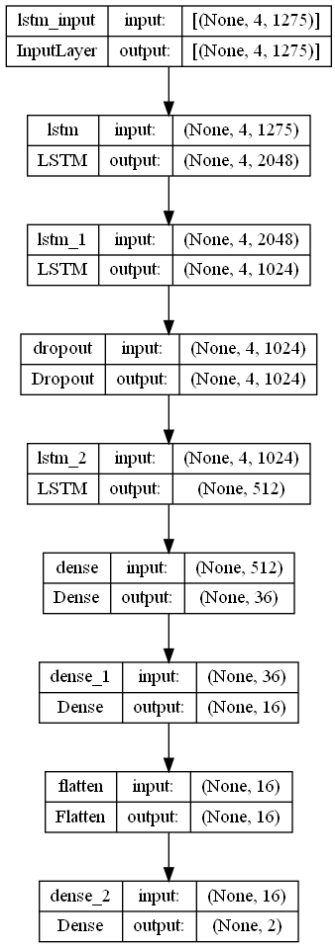

Fig. 8. LSTM architecture where the input shape varies by biosignal. EEG: (length of sample, # of channels) EMG: (length of sample, 1)

- **GRU**: The Gated Recurrent Unit (GRU) is a relatively recent development compared to the LSTM. The GRU is similar in design to the LSTM unit with some differences. GRUs lacks memory units and instead just contain a reset and update gate. Per iteration, instead of containing a hidden state like the LSTM, the GRU exposes its entire state. Due to the simpler design, the GRUs are expected to perform more efficiently than LSTM. We implement the GRU by taking the LSTM architecture described earlier and swapping the LSTM units with GRU units.

B. Training and Evaluation

For each model in our experiments, we employ Grid Search to find the optimal set of hyper-parameters: learning rate and

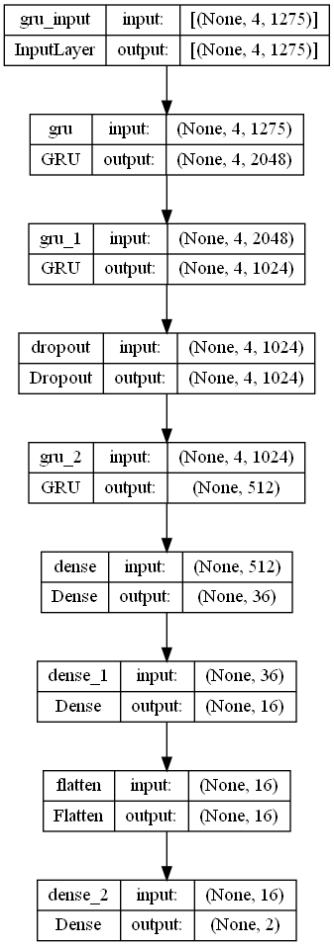

Fig. 9. GRU architecture where the input shape varies by biosignal. EEG: (length of sample, # of channels) EMG: (length of sample, 1)

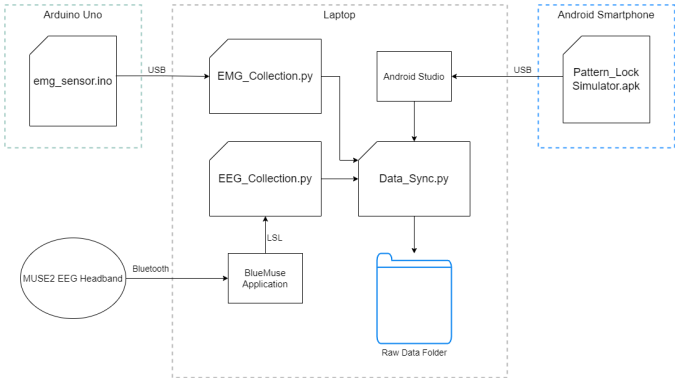

Fig. 10. Diagram of the data collection orchestration and code solution

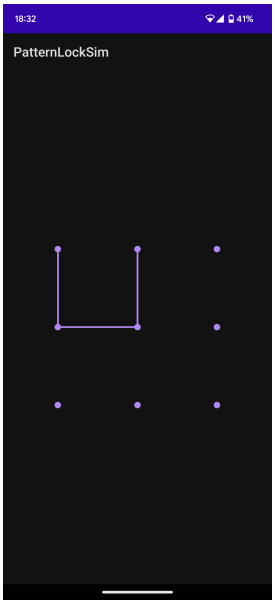

Fig. 11. Screenshot of our Android app to Simulate Android Pattern Lock

dropout. The list of learning rates is 0.01, 0.001, 0.0001 and the list of dropout values tested is 0.0, 0.2, 0.4. Since our dataset is small, we use 5-fold Stratified Cross Validation during training and take the average of each fold's model metrics to evaluate the model. Early stopping is also used during each fold where loss is measured with a patience value of 10 and a delta of 0.001. By default, 200 epochs are used for training if the early stopping threshold is not met.

To evaluate each model, we use the following set of model metrics: the average accuracy, recall, and run-time of the model. To analyze the classification quality, we create confusion matrices to analyze how well the models predicted the classes on the test sets.

VI. Study Implementation

In this section, we provide the implementation of software components used in our experiment such as data collection, data processing, and data classification. We use Java and Python along with third-party libraries to develop and test our software. First, we describe the data collection solution outlined in Figure 10, and then go into detail on the separate code solution created for model creation, training, and evaluation.

A. Android Pattern Lock data collection

The first part of conducting our experiment is providing a way for users to repeatedly enter Android Pattern Locks to build a dataset. To do this, we developed an Android app which simulates the stock Android Pattern Lock interface and allows for us to log the patterns drawn by the user on this interface. The app, shown in Figure 11 is made on Android Studio using Java. To simulate the Android Pattern Lock interface, the *com.andrognito.patternlockview* library was used. This library provides a *PatternLockView* component which we add to the layout XML file so that the app displays the 3x3 Android Pattern Lock grid. The library also provides a class and listener which we use to catch and handle events such as when a single pattern has started being drawn and when that pattern is finished being drawn. Each event is caught be a separate handler within the listener. The listener is implemented in our app's main activity class so that events are caught for the duration that the app is used. A log statement is placed inside

the pattern completion handler where it logs the drawn pattern data to the Android Studio Logcat running on the host laptop for data collection. This streamed data consists of what pattern was drawn and at what timestamp.

B. Biosignal Data Collection

To record biosignal data, EEG and EMG, from our subject, we developed a data collection system consisting of multiple sensors streaming data to the laptop used for data collection as mentioned earlier. EEG and EMG data are recorded separately via separate Python scripts that run in parallel. To run the scripts in parallel, the PyCharm IDE's Compound configuration is utilized. The amount and type of trials conducted per session is controlled by the EEG recording script. Once the session terminates, the script prints an exit command to the console which safely terminates the EMG script. In this section we explain in detail the role of both the EMG and EEG data collection solutions.

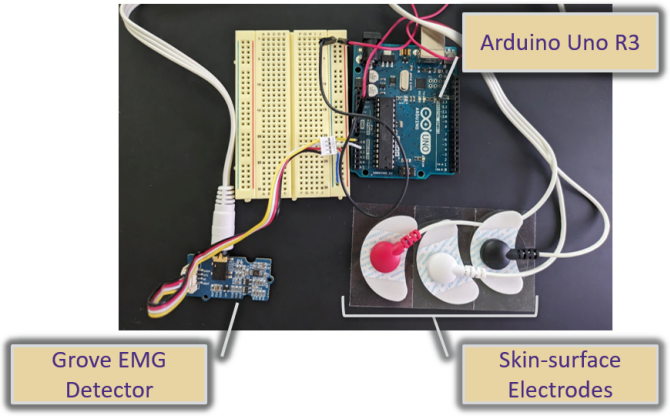

Fig. 12. EMG Data Collection Apparatus with the Arduino (left) and Grove EMG detector (middle) and skin-surface electrodes (right)

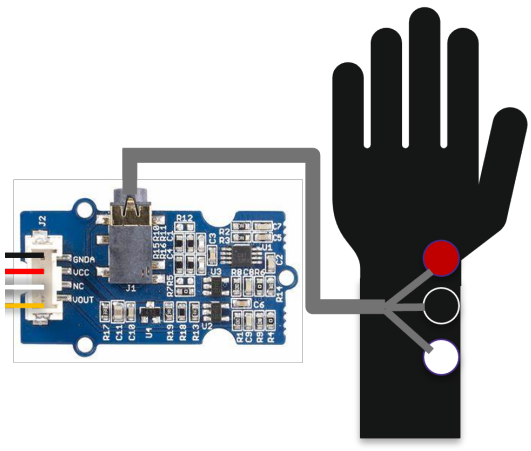

Fig. 13. Skin-surface electrode placement on the hand with the electrodes wired to the Grove EMG detector (left)

- **EEG**: The EEG data comes from a Muse2 EEG Headband which records 4 EEG channels (AF7, AF8, TP9, TP10) at 256 Hz. The Muse2 is connected to the laptop via Bluetooth. The BlueMuse Windows 10 app and muselsl Python libraries are used to stream and save EEG data from the Muse2 into .csv files.
- **EMG**:The EMG data comes from skin-surface electrodes placed on the wrist as shown in 13. The electrodes are connected to an Arduino via a Grove EMG detector as shown in Figure 12. The Arduino transmits EMG data to the laptop via serial connection at a baud rate of 11520. The serial Python library is used to read incoming data and save it into .csv files.

1) EEG Data: The EEG data collection script's role is to manage a data recording session and collect EEG data from the user. This script takes in arguments to configure the pattern classes or line direction classes being collected and at what quantity (n). A multi-nested for loop is used to orchestrate the session, where a list of pattern classes or line direction classes is traversed. The user is prompted to draw the currently selected class for n-number of times. Next, the Python *muselsl* package is used to open a LSL stream with the connected MUSE2 EEG Headband via the BlueMuse Windows application. EEG data is then recorded for 5 seconds and saved to a .csv file. The contents of this .csv file includes 5 columns: a column per channel that the MUSE2 records (AF7, AF8, TP9, TP10), and a column for the timestamps for each sample of the 5-second signal. After the entire session ends, an exit command is printed to the console which terminates the corresponding EMG recording solution and concludes recording of all biosignal data. Since this script orchestrates the recording session, it keeps track of labeling the samples by including the label in the filename of the .csv file.

2) EMG Data: The EMG recording solution comprises of two software components, the first of which runs on our Arduino and the second runs on our data collection laptop. The first component is Arduino software which reads analog data from the connected Grove EMG Detector and prints it to the serial port at a baudrate of 115200. This is performed using the built-in *Serial.read()* and *Serial.println()* methods in the Arduino programming language.

The second component is a Python script which uses the *serial* package to read serial data from the Arduino's comport at the same baudrate of 115200 and decodes the incoming bytes. This logic is implemented inside of a while loop so that EMG data is continuously read from the Arduino and saved into a Python dataframe. The dataframe is later saved as a .csv file once the exit command is received from the EEG data collection script as mentioned earlier.

C. Dataset Creation

Since the EEG and EMG data is stored in separate locations, a sync tool is used to combine the data into a single sample for ease of use in later steps. The syncing process executes after each data collection session, where it is assumed a single .csv

file of EMG data and multiple smaller .csv files of 5-second-long EEG data are newly created as described in the previous section. This tool is implemented as a Python script which traverses the directory storing the EEG recordings and grabs newly added recordings from the newest session. For each recording, the *merge_asof()* function from the Python *pandas* package is used to do a left-join on the timestamp column of the EEG and EMG dataframes (where EEG data is on the left of the join and EMG data is on the right of the join). The result is a single dataframe with synced data which represents the 5 seconds of EEG and EMG data of the user as they drew a pattern or line direction. The dataframe is saved as a new .csv file and kept in a separate directory for later use with the same name as the original .csv file to keep track of class labeling. The result of this is a folder structure where each folder represents a class that contains samples. Each sample is represented by a .csv file with EEG and EMG data.

D. Data Processing & Feature Extraction

After creating the dataset, we perform a set of analysis steps on the EEG and EMG data using a Python script named Dataset_Processing.py. The Python *os* package is used to traverse the dataset and the *pandas* library is used to read each .csv file as a Python dataframe. Per sample, we ensure it is a valid 5-second-long sample by filtering out samples which have null values or are of the wrong length. We found and removed a handful of erroneous samples which did not match the criteria. The analysis is different for EEG and EMG data. For EMG data we normalize the signals. For EEG data, we use the *mne* package to apply filters to EEG data to keep the Beta frequency band (13 - 30 Hz.) and remove the 50Hz. powerline noise.

E. Model Creation

We create a separate Python script named Model_Creation.py to represent the different models that we experiment with, 1D-CNN, LSTM and GRU. This script has functions which take in parameters like learning rate, dropout rate, and input resolution and return a built model to the evaluation script. We use the *keras* package to build the base model architectures. A function constructs the 1D-CNN and another creates a LSTM or GRU. Both functions initiate the model by creating a *Sequential* object from the *keras* package. The function creating the 1D-CNN adds a series of *Conv1D*, *LeakyReLU*, *MaxPool1D* layer objects to the Sequential object to construct the CNN architecture. *Dense* and *Flatten* layer objects are used to create the last fully connected and output layers. The function creating the LSTM or GRU adds a series of *LSTM* or *GRU* layer objects and has Dense and Flatten layer objects to the Sequential object. For all models, the optimizer is created by constructing the *Adam* object from the *keras.optimizers* class. Refer to Appendix B for more detail on all three of the model architectures.

F. Model Evaluation

Model evaluation is implemented as a Python script which uses the previously mentioned scripts to orchestrate the experiments as shown in Figure 14. First, this script takes in arguments to represent which experiment to run from the three possible experiments (Pattern Lock Detection, Pattern Lock Identification, and Line Direction Classification). The correct processed dataset for the experiment is retrieved using the dataset creation and dataset analysis scripts. Then grid search for hyperparameter tuning and cross validation is implemented as a multi-nested for-loop. The for-loop traverses lists which contain values of learning rates and dropout rates to test. This is the grid search portion of the multi-nested for-loop. The second-innermost loop creates a *KFold* object from the *sklearn* package. The *split* function on the KFold object is used to create index masks. The innermost loop traverses a list of these masks and uses each mask to select the train and test folds from the overall dataset. In the same loop, a model is selected from the model creation script, trained on the training set and evaluated on the test set. The *sklearn* package is used to evaluate the model. The metrics are stored in a list and printed to an output file for analysis.

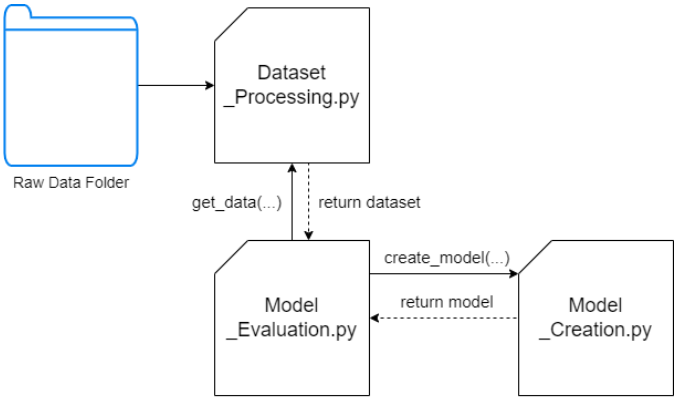

Fig. 14. Diagram showing the relationships between the dataset processing, model creation, and model evaluation scripts.

VII. Results

In this section, we present the results for each of the proposed models during our experiment with hyper-parameter tuning.

A. Pattern Lock Detection

For Pattern Lock Detection, we analyze the results from the models trained for binary classification of the EEG signals. Table I shows the hyperparameter tuning of the models and which combination yields the best results. The CNN performed the best with a accuracy of 96.97% when trained with a dropout of 0.4 and a learning rate of 0.0001. The LSTM and GRU had a noticeably less accuracy score than the CNN model. This is likely due to the convolution layers of the CNN model being better at reducing the dimensionality of the highly-dimensional EEG data and extracting local features. However, the LSTM and GRU performed similarly to each other which is likely due to similar model designs. Certain combinations of learning rate and dropout cause either the

LSTM or GRU to outperform the other. Overall, it is feasible to train a deep learning model on a users EEG signals to discern between the user doing an Android Pattern Lock actions or doing random scrolling or swiping actions.

TABLE I
COMPARISON OF CLASSIFICATION ACCURACY BASED ON LEARNING RATE AND DROPOUT RATE FOR PATTERN LOCK DETECTION

	Learning Rate: 0.01	Learning Rate: 0.001	Learning Rate: 0.0001
No Dropout	CNN: 91.15% GRU: 65.36% LSTM: 81.47%	CNN: 96.67% GRU: 95.20% LSTM: 87.44%	CNN: 92.19% GRU: 83.59% LSTM: 85.55%
Dropout: 0.2	CNN: 86.22% GRU: 64.28% LSTM: 80.75%	CNN: 96.49% GRU: 88.97% LSTM: 90.67%	CNN: 96.91% GRU: 84.48% LSTM: 86.58%
Dropout: 0.4	CNN: 95.76% GRU: 75.74% LSTM: 84.99%	CNN: 96.97% GRU: 89.54% LSTM: 91.05%	CNN: 96.73% GRU: 85.85% LSTM: 87.66%

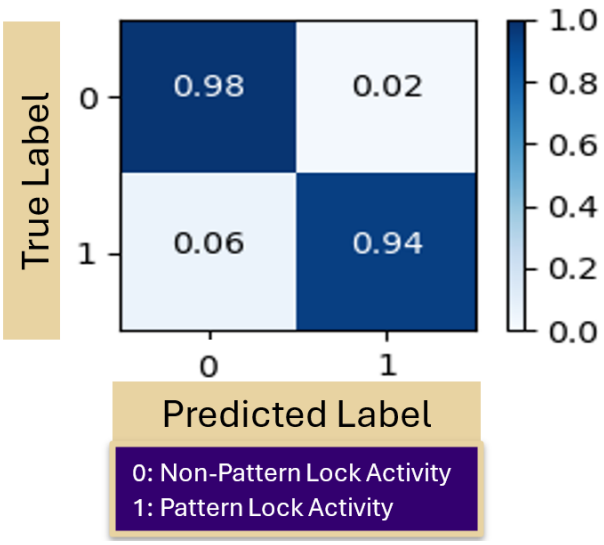

Fig. 15. Confusion Matrix of 1D CNN model for Pattern Lock Detection

From the confusion matrix in Figure 15, we can calculate the precision and recall values. The CNN had a precision of 0.979 and a recall of 0.94 which is strong and shows that there were minimal false negative and false positive predictions. In addition the CNN could discern well between non pattern lock activity and a true pattern lock activity. This shows that the 1D CNN model can extract relevant features from the EEG data, that relate to the activity of doing a Pattern Lock activity. Thus, the user activity such as whether they are using the phone for Pattern Lock or not can be inferred with good accuracy from the EEG signal.

Looking at the training run-time for each of the models in Figure 16, we can see that the CNN trained the quickest while the GRU and LSTM took longer to train. The CNN is likely quicker to train due to its design where convolution layers are more efficient than the LSTM or GRU units. This allowed the CNN to extract relevant features from the EEG data and converge quicker. From the perspective of our threat model, this is an advantage since a CNN model could be developed to efficiently differentiate Pattern Lock activity from random phone activity such as scrolling and swiping.

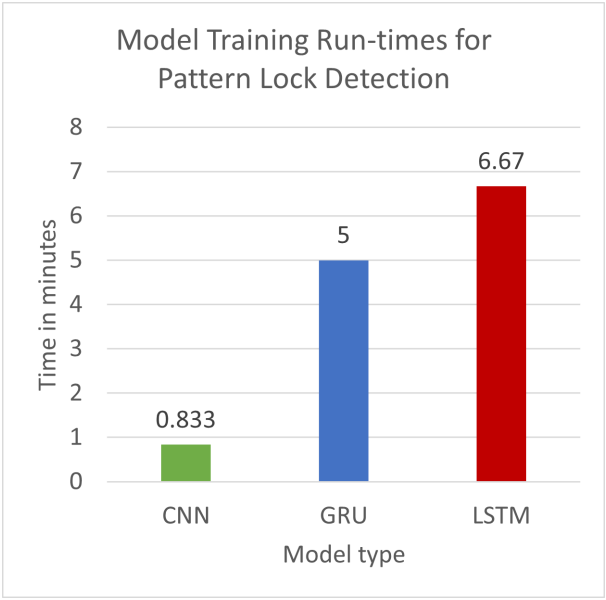

Fig. 16. Training time comparison for the CNN, GRU, and LSTM models for Pattern Lock Detection

B. Pattern Lock Identification

For Pattern Lock Identification, we analyze the results from the models trained for multi-class classification on the EMG signals. The results are described for the two separate experiments for pattern Pattern Lock Identification: inferring predetermined patterns and inferring all possible vectors.

1) Predetermined Pattern Locks: In this task, we had users draw randomly generated patterns of varying complexity on the standard 3 x 3 Android Pattern Lock grid. Table II lists the results of hyperparameter tuning on the LSTM model.

TABLE II
COMPARISON OF CLASSIFICATION ACCURACY BASED ON LEARNING RATE AND DROPOUT RATE FOR PATTERN LOCK IDENTIFICATION ON 6 PREDETERMINED PATTERNS

	Learning Rate: 0.01	Learning Rate: 0.001	Learning Rate: 0.0001
No Dropout	CNN: 65.28% GRU: 90.07% LSTM: 95.07%	CNN: 93.11% GRU: 97.93% LSTM: 96.02%	CNN: 91.45% GRU: 96.17% LSTM: 96.23%
Dropout: 0.2	CNN: 68.98% GRU: 91.36% LSTM: 95.80%	CNN: 95.86% GRU: 98.49% LSTM: 96.91%	CNN: 92.99% GRU: 97.96% LSTM: 98.98%
Dropout: 0.4	CNN: 72.54% GRU: 92.22% LSTM: 96.67%	CNN: 97.72% GRU: 99.97% LSTM: 97.20%	CNN: 94.35% GRU: 99.07% LSTM: 98.68%

The GRU model trained with a learning rate of 0.001 and a dropout of 0.4 performed the best out of all the other combinations. The LSTM model has almost the same

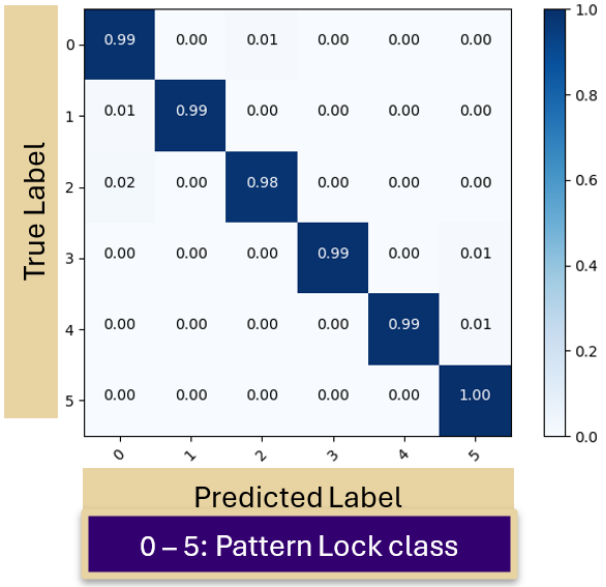

Fig. 17. Confusion Matrix of the GRU model for Pattern Lock Identification on 6 Predetermined Patterns

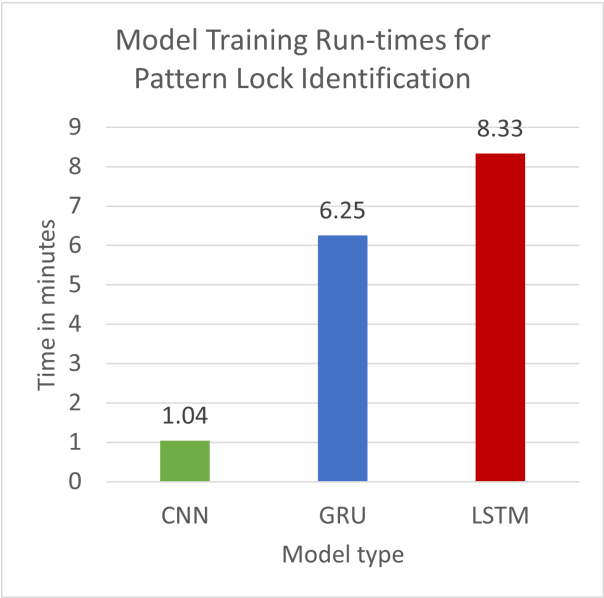

Fig. 18. Training time comparison for the CNN, GRU, and LSTM models

performance as the GRU which can be expected since the design of their model units are similar. The CNN performed decently when the learning rate was above 0.01 and had a low run-time. Overall, the models designed for classifying sequential data are better suited for the Android Pattern Lock identification inference problem. As the GRU performs the best, we observe how it classified each pattern class by creating a confusion matrix. Overall, the results how that it is feasible for a user's unlock pattern to be inferred by analyzing their EMG data.

The confusion matrix of the GRU model shown in Figure 17 reveals that it had a strong average recall of 0.99. This means that it is feasible for a user's unlock pattern to be inferred by analyzing their EMG data. The training times for the models shown in Figure 18 shows similar results to Pattern Lock detection where the CNN trained the quickest. The GRU has a clear advantage over the LSTM where it has a higher accuracy than the LSTM and trains 25% quicker. In our side-channel attack using the GRU is the most ideal option. Using the CNN could be an advantage if training had to take place in around a minute. However, this would be at the expense of model accuracy.

2) All Possible Pattern Lock Line Directions: In this task, users drew all the possible line directions for the standard 3 x 3 Android Pattern Lock grid using our Android Pattern Lock simulator. Table III details the results of hyperparameter tuning of all the models implemented. The best combination was the LSTM with trained with no dropout with the smallest learning rate of 0.0001. Typically a decrease in accuracy is not expected when increasing the dropout values. But increasing the dropout can lead to a more simplistic models as neurons are dropped. This can lead to information loss during training and a slight decrease in classification accuracy which can explain the result. The GRU performed similarly to the LSTM, while the CNN did not perform as well. However, when keeping the learning rate at 0.01 for all three models, the CNN performs significantly better than the LSTM and GRU. Overall, the models designed for sequential classification, LSTM and GRU, performed better for inferring all 16 possible directions on the 3x3 Pattern Lock grid.

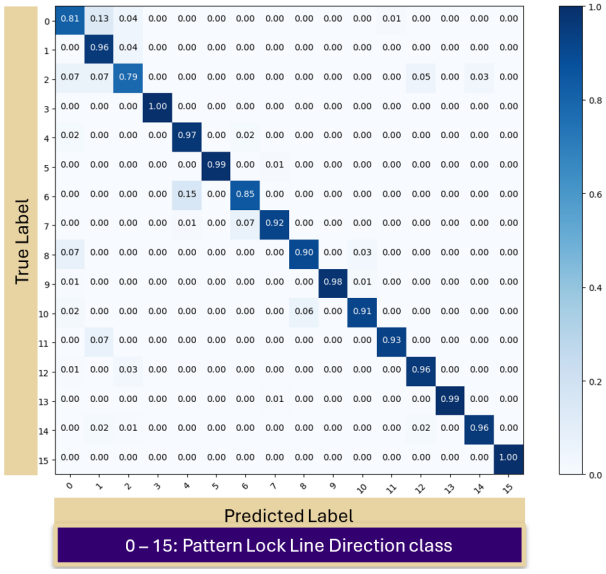

Fig. 19. Confusion Matrix of LSTM model trained for Pattern Lock Identification on all Possible Line Directions

The confusion matrix shown in Figure 19 shows that the LSTM model classified some classes of directions better than others. For instance, the directions on the right side of the grid (labelled 0 - 7) were classified worse than the directions on

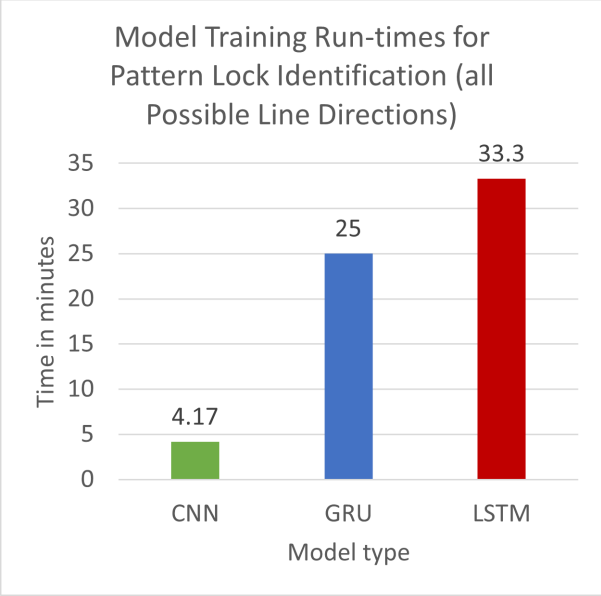

Fig. 20. Training time comparison for the CNN, GRU, and LSTM models

TABLE III
COMPARISON OF CLASSIFICATION ACCURACY BASED ON LEARNING RATE AND DROPOUT RATE FOR PATTERN LOCK IDENTIFICATION ON ALL POSSIBLE LINE DIRECTIONS

	Learning Rate: 0.01	Learning Rate: 0.001	Learning Rate: 0.0001
No Dropout	CNN: 70.01% GRU: 32.24% LSTM: 37.12%	CNN: 83.49% GRU: 91.04% LSTM: 88.68%	CNN: 76.39% GRU: 91.50% LSTM: 93.64%
Dropout: 0.2	CNN: 72.45% GRU: 33.96% LSTM: 35.87%	CNN: 75.39% GRU: 92.72% LSTM: 84.20%	CNN: 78.71% GRU: 92.33% LSTM: 91.92%
Dropout: 0.4	CNN: 74.31% GRU: 31.99% LSTM: 31.41%	CNN: 77.84% GRU: 92.28% LSTM: 80.54%	CNN: 80.73% GRU: 92.08% LSTM: 90.25%

the left side of the grid (8 - 15). This could be because we collected data only from a user who used their right thumb to draw on the grid. The right thumb extension may be more pronounced for movements on the left or far side of the grid. This could lead to a higher fidelity EMG signal than directions drawn on the right side of the grid. Nonetheless, the LSTM model had a precision of 0.931 and a recall of 0.933 which is strong. These results show that a future system can utilize this LSTM model to infer Android Pattern Lock by recognizing individual line segment directions that make up the drawn pattern.

This classification problem increased the training times for all three models significantly as shown in Figure 20. The CNN managed to train in 4.17 minutes which is quick compared to the training times of the GRU and LSTM, which took 25 and 33.3 minutes, respectively. The rise in training time is due to the increased complexity of classifying the 16 different line directions that one's finger can draw on the 3x3 Pattern Lock grid.

C. Comparison to Related Works

Our work exists at the crossroads between biosignal-based side side channel attacks and cracking alternative authentication mechanisms such as Android Pattern Lock. This work applies both EMG and EEG signal analysis techniques to create a novel biosignal side-channel attack to infer Android Pattern Lock. Prior work has cracked Android Pattern Lock using different side channels or techniques from other domains [35], [37]. Since their evaluation methods are not similar to ours, we cannot make a direct comparison. However, we attempt to compare the closest metrics and the best metrics from the related works in separate comparisons. The video-based attack [35] and the acoustic side-channel attack [37] measure the percentage of patterns cracked under a varying number of attempts, between 1 and 5 attempts. We measure the accuracy of our deep learning models when predicting the unlock pattern from a set of unseen EMG signals labelled based on the unlock pattern drawn at the time of recording. The model accuracy is the percentage of EMG signals categorized correctly out of all the EMG signals in the test set. Our model could correctly infer each of the unlock patterns with a certain accuracy. The model's overall accuracy on classifying the unlock patterns is an average of those accuracy values as shown in our results. Since the prediction on the test set happens once, the closest comparable metric to model accuracy is the percent of patterns cracked in 1 attempt which is shown in Figure 22.

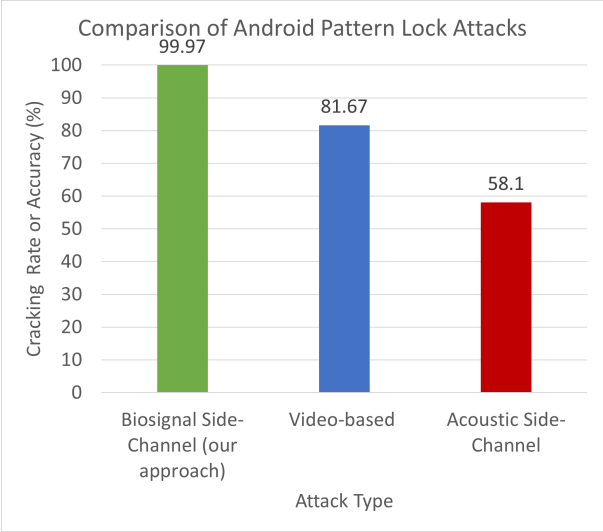

Fig. 21. Comparison of our accuracy to the cracking rate (within 1 attempt) for a video-based attack [35] and an acoustic side-channel attack [37].

For completeness we also compare the best metrics from the other attacks to our model accuracy. Figure 21 shows how effectiveness of the other attacks increases with more attempts. Other differences in our evaluation also exist. Each of the Android Pattern Lock works, including ours, evaluated their methods on a different set of unlock patterns. Apart from cracking unlock patterns, our work provides results for

additional aspects of cracking Android Pattern Lock. This work is likely the first to detect when the victim was using Android Pattern Lock based on their brain activity, and also classify the 16 unique directions draw-able on the 3x3 grid.

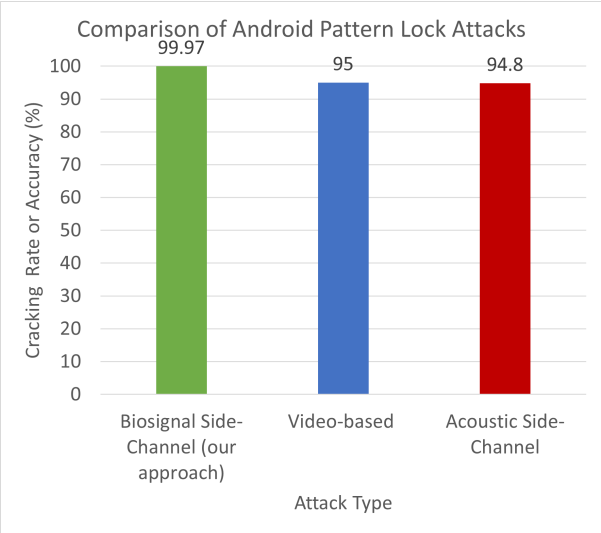

Fig. 22. Comparison of our accuracy to the cracking rate (within 5 attempts) for a video-based attack [35] and an acoustic side-channel attack [37].

1) Biosignal-Based Attacks: To our knowledge, our work is likely among the first to analyze EMG and EEG signals to infer the activity of using Android Pattern Lock. However, this work is inspired by prior works which have showcased using biosignals such as EEG and EMG separately to infer similarly sensitive information. Specifically, work done to use EMG to infer keystrokes [9] and to use EEG to infer PINs [25]. Our work differs to those works and existing works on BCI side channel attacks by introducing the Android Pattern Lock attack scenario and using both EMG and EEG signals in a combined inference approach. Furthermore, our work also differs in the usage of the EEG and EMG signals. The work done to infer keystrokes [9] using EMG used expensive MyoWare multi-channel EMG armbands to infer muscle movements of the arm, wrist and hands. Our work focused only the thumb muscle activity and collected data using a circuit made with hobby-level skin-surface EMG electrodes and an Arduino. For collecting and using EEG signals, we used a Muse2 EEG Headset which is an affordable, commercially available device. The work by Neupane et al. [25] compared two headsets, one being the Emotiv Epoch Headset which is a more expensive commercially available device, and the B-Alert Headset which is a clinical-level headset.

2) Computer Vision-Based Attacks: Previous work have demonstrated video-based attacks in decoding Android Pattern Lock, where the attacker records the victim drawing their pattern and attempts to predict the victim's finger position on their phone screen [35]. This method could crack 95% of their patterns within 5 attempts and on average 81.67% of patterns within a single attempt. These metrics do not consider patterns that couldn't be classified due to a blurred video. This attack relies on knowledge that the victim is unlocking their phone via Android Pattern Lock and that the recorded video has decent quality, lighting, and an unobstructed view of the victim's finger as they draw their unlock pattern. Their pattern cracking rate decreased for simpler patterns whereas our classification accuracy went up for similar patterns.

3) Acoustic Signal-Based Attack: A different type of side channel attack, an acoustic attack was devised to decode Android Pattern Lock [37]. The acoustic attack works by using the phone's microphone to gather imperceptible sound waves that are generated by the phone's speakers and are reflected off of the user's fingers as it moves on the screen. This method yields a pattern cracking rate of 94.8% of patterns cracked within 5 attempts and 58.1% of patterns cracked within 1 attempt. But this attack relies on the user to have their microphone enabled for background processes which can no longer be covertly done in modern versions of Android where the phone user is notified of microphone usage by a service or app.

4) Physical-Access Attacks: Besides side-channel attacks, attackers can also decode the Android Pattern lock through other means involving physical access to a victim's device or device hard-drive. The Android Operating System encrypts user's unlock patterns (represented as a sequence of integers) with the SHA1 hashing algorithm and stores it in a gesture.key file on the system directory [28]. Thus, an attacker with access to this file could decrypt the unlock pattern if they have a dictionary of all possible pattern lock sequences and their SHA1 hash values. This attack is dependent on the attacker having access to the victim's phone to access the gesture.key file. This file can only be accessed if the phone's OS has root access or USB debugging enabled, and the attacker has a PC on hand to view the file. This is very unlikely as the general public don't use their phones in this state. Another attack involves observing the oily residue that is left over from swiping a finger across the screen to draw the unlock pattern [3]. This attack can be very effective but the smudges on the screen may not always be intact, or they can be distorted by other finger swipes or by the user placing their phone back in their pocket.

VIII. Discussion

In this section, we outline potential countermeasures and the strengths and limitations of our proposed side channel attack.

A. Potential Countermeasures

The success of our side channel attack requires that the attacker accesses the victim's EEG and EMG signals via covert means such as a malicious BCI or HCI application. A potential method to counter this is to make the act of recording biosignal data less covert. Similar to how the latest versions of Android informs the user of microphone and video recording activities, notifications can be pushed to the user's smartphone when biosignal data is being recorded by third-party applications.

The user can terminate those processes to ensure their biosignal data isn't being leaked when they are performing sensitive activities. However, this mitigation is contingent on the user having the awareness to act when receiving such notification. Apart from notifications, the processes recording biosignal data can be halted based on the current phone activity. For example, halting biosignal recording when it is detected that the user is unlocking their phone. Without the EMG data, it would make it difficult for the hacker to gauge the exact entry pattern drawn. Another approach is adding noise to the signal at all times. The downside to both of these approaches is that interrupting biosignal recording negatively impacts non-malicious applications that rely on high fidelity biosignal data. This may have adverse implications for users that rely on biosignal data for health monitoring and diagnosing conditions. In general, more secure software on the applications that interface with BCI or EMG-based HCI devices is paramount to mitigating our attack scenario.

B. Strengths and Limitations

Our study has several strengths and some limitations. Data collection was conducted in a semi-realistic setting where the human participant was drawing unlock patterns on a realistic Android Pattern Lock interface. In addition, the EEG and EMG data was recorded using devices and hardware readily available to everyday consumers. We also compared a variety of deep learning models and hyperparameters to properly process and analyze the biosignal data. However, limitations with our approach exist. We could only collect EEG data from a single participant who only used their right thumb to draw the unlock patterns. Furthermore, due to our relatively small dataset size, cross validation was leveraged to validate that our models perform well. Having multiple participants would allow for a more diverse dataset and the ability to gauge if our EEG and EMG processing and analysis methods are generalized enough to a broader population.

Despite the limitations, we believe that our study highlights the feasibility of a side-channel attack to infer unlock patterns. In addition, our study brings awareness to vulnerabilities present in EEG headsets and potential EMG wristbands and the risks associated with leaking biosignal data. Our study accomplishes this by showcasing how biosignal processing steps and deep learning models can be built to discern various Android Pattern Locks from EEG and/or EMG data from a subject.

C. Future Work

While our work shows promising results, addressing some of the limitations and further evaluating our methods would require future study. As stated earlier, more data collection from a diverse set of subjects would allow us gauge the effectiveness of our data processing and analysis techniques on a larger sample size, more representative of a population. Addressing this limitation is feasible since our current data collection and processing implementation can scale to account for multiple participant's biosignal data. Applying and comparing more biosignal data processing techniques such as converting from time domain to the frequency domain would allow us to further denoise the data and identify hidden features. Furthermore, by converting to frequency domain, we can compare EEG signals across various subjects and conditions from our expanded, diverse dataset. In addition, as demonstrated by the results of our experiment to classify all 16 draw-able vectors on a 3x3 grid, our biosignal side-channel attack can be extended to potentially infer all possible unlock patterns by classifying each component of the drawn pattern.

IX. Conclusion

In this paper, we have provided deep learning approaches to demonstrate the side-channel attack scenario of potentially using EEG and EMG data to infer a user's Android Pattern Lock activity. We studied two sub inference problems in this process, detecting the activity of doing Pattern Lock and identifying the pattern drawn while doing the Pattern Lock activity. In this process, we created a recording apparatus and compiled a dataset of EMG and EEG signals. We implemented a 1D CNN model which could detect the user activity of doing Pattern Lock from their EEG signals with 96.97% accuracy. We also implemented an LSTM model to infer the pattern drawn during Pattern Lock from their EMG signals with a 99.97% accuracy. We then demonstrated that the same LSTM model can be used in a future system to detect all of the possible line directions drawable on the standard 3 x 3 Pattern Lock grid, with 93.64% accuracy.

The popularity and accessibility of advanced health-related IoT devices continue to rise as more consumers realize the benefits of monitoring their bioignals. Hence, it is important to continue future studies on the security implications of having EEG or EMG data intercepted from vulnerable EEG headsets, BCIs, and EMG wristbands.

References

[1] A. Agarwal and e. al. Protecting privacy of users in brain-computer interface applications. In *ACM Woodstock conference*, 2019.

[2] M. Aldayel, M. Ykhlef, and A. Al-Nafjan. Deep learning for eeg-based preference classification in neuromarketing. *Applied Sciences.*, 2020.

[3] A. J. Aviv, K. Gibson, E. Mossop, M. Blaze, and J. M. Smith. Smudge attacks on smartphone touch screens. WOOT'10, page 1–7, USA, 2010. USENIX Association.

[4] R. S. Bisht, S. Jain, and N. Tewari. Study of wearable iot devices in 2021: Analysis & future prospects. In *2021 2nd International Conference on Intelligent Engineering and Management (ICIEM)*, 2021.

[5] L. Dunai, M. Novak, and C. G. Espert. Human hand anatomy-based prosthetic hand. *Sensors*, 21(1):137, Dec 2020.

[6] S. Eberz et al. When your fitness tracker betrays you: Quantifying the predictability of biometric features across contexts. In *Proc. IEEE Symp. Security Privacy (SP)*, 2018.

[7] B. Fan, X. Liu, X. Su, P. Hui, and J. Niu. Emgauth: An emg-based smartphone unlocking system using siamese network. In *2020 IEEE International Conference on Pervasive Computing and Communications (PerCom)*, pages 1–10, 2020.

[8] T. Flavin, T. Steiner, J. Reaves, J. Abhari, Z. Delap, B. Mitra, and V. Nagaraju. Ml democracy: An enhanced voting algorithm for model selection for efficient eeg data assessment. In *2022 21st IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 1815–1820, 2022.

[9] M. Gazzari, A. Mattmann, M. Maass, and M. Hollick. My(o) armband leaks passwords: An emg and imu based keylogging side-channel attack. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 5(4), dec 2022.

[10] F. Heinrichs, M. Heim, and C. Weber. Functional neural networks: Shift invariant models for functional data with applications to eeg classification. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ICML'23. JMLR.org, 2023.

[11] R. Hiroki and M. Iwase. Hand and finger control of myo-prosthesis based on motion discriminator and voluntary control. In *2017 11th Asian Control Conference (ASCC)*, pages 1361–1366, 2017.

[12] L. Hyunin, K. Dongwook, and P. Yong-Lae. Explainable deep learning model for emg-based finger angle estimation using attention. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, 30:1877–1886, 2022.

[13] A. L. Trejos J. Tryon. Evaluating convolutional neural networks as a method of eeg–emg fusion. *Frontiers in Neurorobotics*, 15, 2021.

[14] M. Kapitonova, P. Kellmeyer, S. Vogt, and T. Ball. A framework for preserving privacy and cybersecurity in brain-computer interfacing applications. *Innovation for Cybersecurity*, 2022.

[15] D. Y. Kim, D. K. Han, J. H. Jeong, and S. W. Lee. Eeg-based driver drowsiness classification via calibration-free framework with domain generalization. In *2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 2293–2298, 2022.

[16] B. J. King, G. J. M. Read, and P. M. Salmon. The risks associated with the use of brain-computer interfaces: A systematic review. *International Journal of Human–Computer Interaction*, 2022.

[17] J. Kowaleski. Bluemuse. https://github.com/kowalej/bluemuse. 2021.

[18] E. Lashgari, D. Liang, and U. Maoz. Data augmentation for deep-learning-based electroencephalography. *Journal of Neuroscience Methods*, 346:108885, 2020.

[19] K. H. Lee, J. Y. Min, and S. Byun. Electromyogram-based classification of hand and finger gestures using artificial neural networks. *Sensors*, 22(1):225, Dec 2021.

[20] Q. Li, D. Ding, and M. Conti. Brain-computer interface applications: Security and privacy challenges. In *IEEE Conference on Communications and Network Security*, 2015.

[21] Y. LI, J. Huang, F. Tian, H. A. Wang, and G. Z. Dai. Gesture interaction in virtual reality. *Virtual Reality & Intelligent Hardware*, 1(1):84–112, 2019.

[22] A. Mandal and N. Saxena. Sok: Your mind tells a lot about you: On the privacy leakage via brainwave devices. *Networks WiSec*, 2022.

[23] Ivan Martinovic, Doug Davies, Mario Frank, Daniele Perito, Tomas Ros, and Dawn Song. On the feasibility of side-channel attacks with brain-computer interfaces. In *Proceedings of the 21st USENIX Conference on Security Symposium*, Security'12, page 34, USA, 2012. USENIX Association.

[24] R. Mishra, K. Sharma, and A. Bhavsar. Visual brain decoding for short duration eeg signals. In *29th European Signal Processing Conference (EUSIPCO)*, 2021.

[25] A. Neupane, M. L. Rahman, and N. Saxena. Peep: Passively eavesdropping private input via brainwave signals. *Financial Cryptography and Data Security: 21st International Conference*, 2017.

[26] F. Orlando. Classification of extension and flexion positions of thumb, index and middle fingers using eeg signal. 11 2016.

[27] V. Ponciano, I. M. Pires, F. R. Ribeiro, N. M. Garcia, M. V. Villasana, E. Zdravevski, and P. Lameski. Machine learning techniques with ecg and eeg data: An exploratory study. *Computers*, 9(3), 2020.

[28] V. V. Rao and A. S. N. Chakravarthy. Analysis and bypassing of pattern lock in android smartphone. In *2016 IEEE International Conference on Computational Intelligence and Computing Research (ICCIC)*, pages 1–3, 2016.

[29] Y. Roy, H. Banville, I. Albuquerque, A. Gramfort, T. H. Falk, and J. Faubert. Deep learning-based electroencephalography analysis: a systematic review. *Journal of Neural Engineering*, 2019.

[30] R. Shahzadi, S. Anwar, F. Qamar, and M. Ali. Secure eeg signal transmission for remote health monitoring using optical chaos. *IEEE Access*, 2019.

[31] C. Spampinato, S. Palazzo, I. Kavasidis, D. Giordano, N. Souly, and M. Shah. Deep learning human mind for automated visual classification. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2017.

[32] D. Valeriani, F. Santoro, and M. Ienca. The present and future of neural interfaces. *Frontiers in Neurorobotics*, 16, 2022.

[33] A. D. Witman, M. C. Brian, and R. G. Avid. Electromyography signal acquisition and analysis system for finger movement classification. *(IJACSA) International Journal of Advanced Computer Science and Applications*, 2019.

[34] Y. Xiao, Y. Jia, X. Cheng, J. Yu, Z. Liang, and Z. Tian. I can see your brain: Investigating home-use electroencephalography system security. *IEEE Internet of Things Journal*, 6(4):6681–6691, 2019.

[35] G. Ye, Z. Tang, D. Fang, X. Chen, K. Kim, B. Taylor, and Z. Wang. Cracking android pattern lock in five attempts. 2017.

[36] J. Zhang, X. Zheng, Z. Tang, T. Xing, X. Chen, D. Fang, R. Li, X. Gong, and F. Cheng. Privacy leakage in mobile sensing: Your unlock passwords can be leaked through wireless hotspot functionality. *Mobile Information Systems*, 2016:1–14, 01 2016.

[37] M. Zhou, Q. Wang, J. Yang, Q. Li, F. Xiao, Z. Wang, and X. Chen. Patternlistener: Cracking android pattern lock using acoustic signals. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, CCS '18, page 1775–1787. Association for Computing Machinery, 2018.

[38] X. Zhou, W. Qi, and S.E. Ovur et al. A novel muscle-computer interface for hand gesture recognition using depth vision. *Journal of Ambient Intelligence and Humanized Computing*, 11, 11 2020.